Technical University of Crete
School of Electrical and Computer Engineering

Academic Year 2025-26
Spring Semester

INF419
Principles of Distributed Systems

Project Report (Phase 1):
Implementation of MapReduce on Kubernetes

Instructor: V. Samoladas

Submitted by:
Stamou Spyridon, 2021030090
Gravalos Georgios, 2021030001
Kerimi Rafaela Aikaterina, 2021030007

Programming Tools

Programming Language & Framework

We have chosen Python to develop all our microservices. For the implementation of our REST APIs, we will use the FastAPI framework, since all 3 of us are already familiar with the framework from past projects. We will test the API endpoints using Postman

Authentication Service

We will use Keycloak for our Authentication Service. It supports OAuth2 and will handle user authentication and generate tokens for Single Sign-On (SSO).

Distributed Data System

We will use SQL, and specifically PostgreSQL as our database. It will reliably store the state of our system, such as job details, task assignments, and worker availability, and allow us to run expressive queries such as "Give me all jobs of user X" , "Give me all tasks of job X and their status", or "Count completed tasks for job X", for monitoring and validation.

Shared File System

We will use MinIO as our shared file system. It is an S3-compatible storage service that will store the initial input data, the intermediate results from the mappers, and the final output files from the reducers.

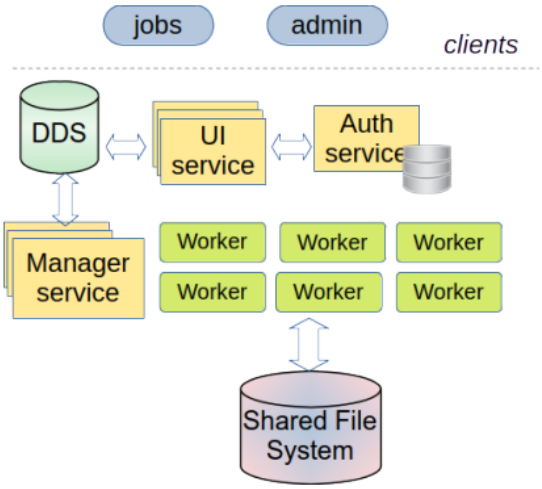
Coding Agents

We will use AI coding agents like Gemini (Antigravity) and ChatGPT to assist us in code generation and overall development of the project.

Orchestration

We will utilize Docker in order to create the images of our services. These services will be orchestrated from Kubernetes using Minikube for our local development.

Microservices



User Interface (UI) Service API

The UI service exposes endpoints to support the two required CLI commands (`jobs` and `admin`):

- `POST /files/upload`: Submits input files to the system. (map/reduce functions and a filename). The endpoint returns the pointers to the files in MinIO.
- `POST /jobs`: Submits the MapReduce job, using references to the previously uploaded input data and mapper/reducer codes.
- `GET /jobs`: Returns the status of existing jobs.
- `GET /jobs/{job_id}` : Return the status of a specific job, not of a list of all jobs.
- `POST /admin/users`: Creates a new user (this is only conceptual, since the user management will be managed through Keycloak)
- `DELETE /admin/users/user_id`: Deletes an existing user (this is only conceptual again, for the same reason)
- `DELETE /jobs/job_id` : Abort a single job ID.
- `POST /admin/nodes`: Configures worker nodes.

Authentication & Authorization Service API

Our system will need:

- the users to log in
- the services to know who the user is and what they are allowed to do within the system
- services to not store passwords themselves
- users not logging in separately with each service (SSO)

For this part, we will not create such a service ourselves. Instead, we will use Keycloak.
Keycloak uses OAuth2 for authorization, and on top of that OpenID Connect for authentication.
When the user wants to connect, the CLI calls Keycloak to provide the user with a signed access token, a JWT.
It will be responsible for managing users, passwords and roles, authenticating users and issuing JWTs, and providing SSO across our microservices.
All internal services validate the JWT access tokens issued by Keycloak and enforce authorization based on the roles contained in the token.
This effectively means that we will not be storing user information (such as username, password hash, email, role, tokens, etc) in a DB of our own, this will be handled entirely by Keycloak, and therefore we will use the endpoints provided by Keycloak.

Manager Service API

The Manager service is an internal service that provides an API for the UI service to schedule jobs:

- `POST /manager/jobs`: Receives job information from the users who use the UI service so that the new job can be scheduled for execution.
- `POST /manager/tasks/status`: Receives status updates from the Workers (e.g., task completed, task failed, or heartbeat) so the Manager can monitor progress and re-assign tasks if a Worker fails.

Workers

Workers are the nodes where all execution happens, specifically running the mappers and reducers. They will be implemented as Kubernetes Jobs running in pods. They do not expose a public REST API.

Distributed Data Service (DDS)

This is our main database. It stores the system's state (such as job details, tasks, and worker status). It does not have a REST API. The Manager service connects to it directly.

Shared File System

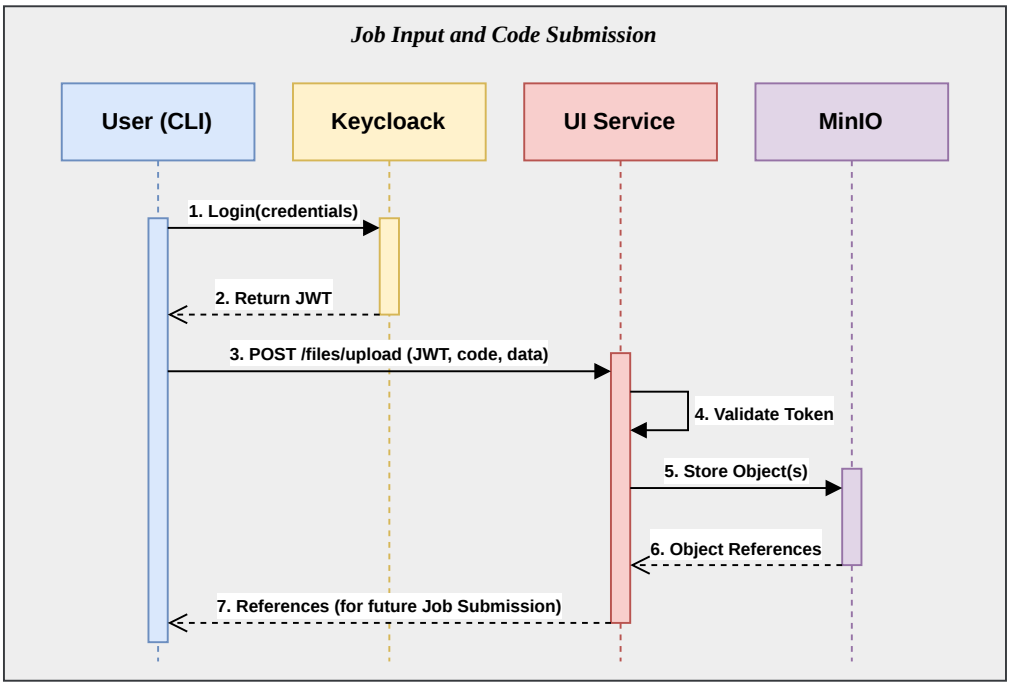
This is the storage space for input and output data. It also does not have a REST API. Workers connect to it directly to read their assigned data and save their results.

Use cases

User submits job input and code

1. The user invokes the CLI to upload the input dataset and the mapper/reducer code to the platform.
2. If the user isn't authenticated, the CLI requests auth through keycloak.
3. Keycloak validates the user credentials, issues a JWT.
4. CLI sends the upload request to the UI service, with the JWT.
5. UI service validates the token, checks that the request is valid. Then, it stores the uploaded dataset and code files in MinIO.
6. MinIO returns the object references to the stored files.
7. UI service returns these references to the client, so they can be included in the job submission request later on.

When this use case is done, the user has been authenticated, the input dataset and the mapper/reducer code files have been saved in MinIO, ready to be referenced by the job execution workflow.



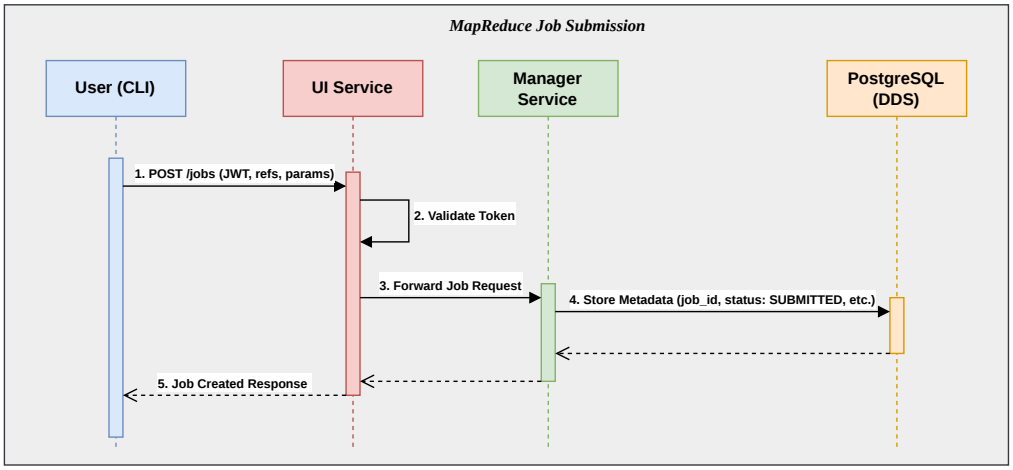
Submission of MapReduce Job

1. User invokes the CLI to submit the job
2. CLI performs authentication, if needed
3. CLI calls `POST /jobs` we mentioned above, with input reference, mapper reference, reducer reference, and any execution parameters (e.g. number of mappers and reducers, etc.)
4. UI validates the token and request, and forwards the request to Manager service.

5. Manager then stores the job's metadata in PostgreSQL, with column fields such as `job_id, owner_user_id, status (e.g. SUBMITTED, RUNNING, COMPLETED, FAILED), input_ref, mapper_ref, reducer_ref, output_ref, created_at, updated_at` and maybe even `err_message` in case of failure.

At this very point, the job exists in the system, but has not been launched yet, either due to limited worker capacity, limited CPU or RAM, scheduling policy, one permitted active job per user or manager replica.

So, the plan for job and task statuses is:
Jobs can be marked as {`SUBMITTED, RUNNING, COMPLETED, FAILED`}, while tasks can be marked as {`PENDING, RUNNING, RETRYING, COMPLETED, FAILED`}



Execution of MapReduce Job

By the time we reach this use case, the job has already been submitted, and its status in the PostgreSQL is `SUBMITTED` (or `QUEUED`, or something similar..)

1. Manager decides to execute the job when the resources are available, or according to job execution policy.

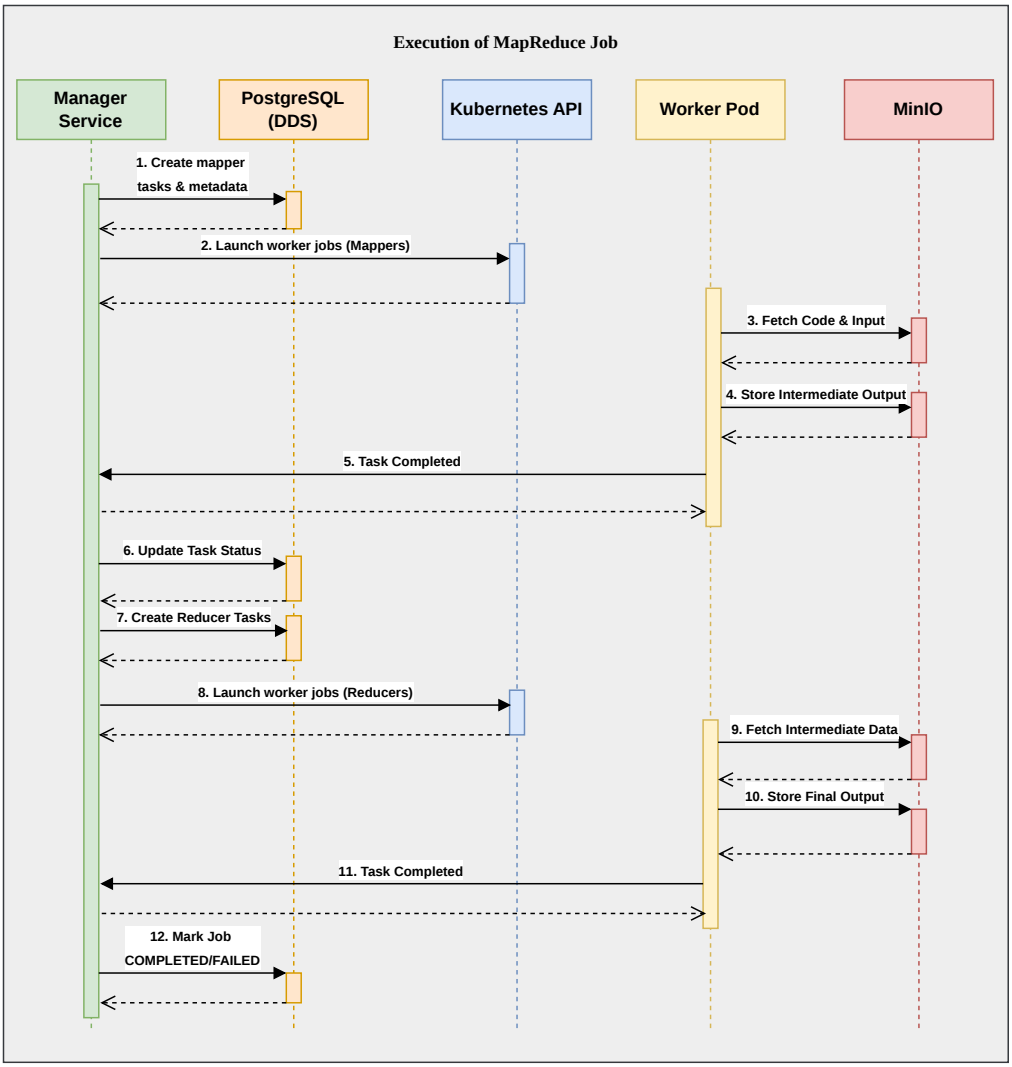
2. Manager creates mapper tasks, records their metadata in PostgreSQL and launches worker jobs in K8s.

3. Workers fetch code and input from MinIO, run the mapper tasks and store the intermediate output in MinIO, while the Manager tracks mapper completion and updates the corresponding task states in PostgreSQL.

4. Once all the mapping tasks are done, the Manager creates reducer tasks, records them in PostgreSQL, and launches the reducer workers.

5. Reducers fetch intermediate data, produce the final output, and store it in MinIO.

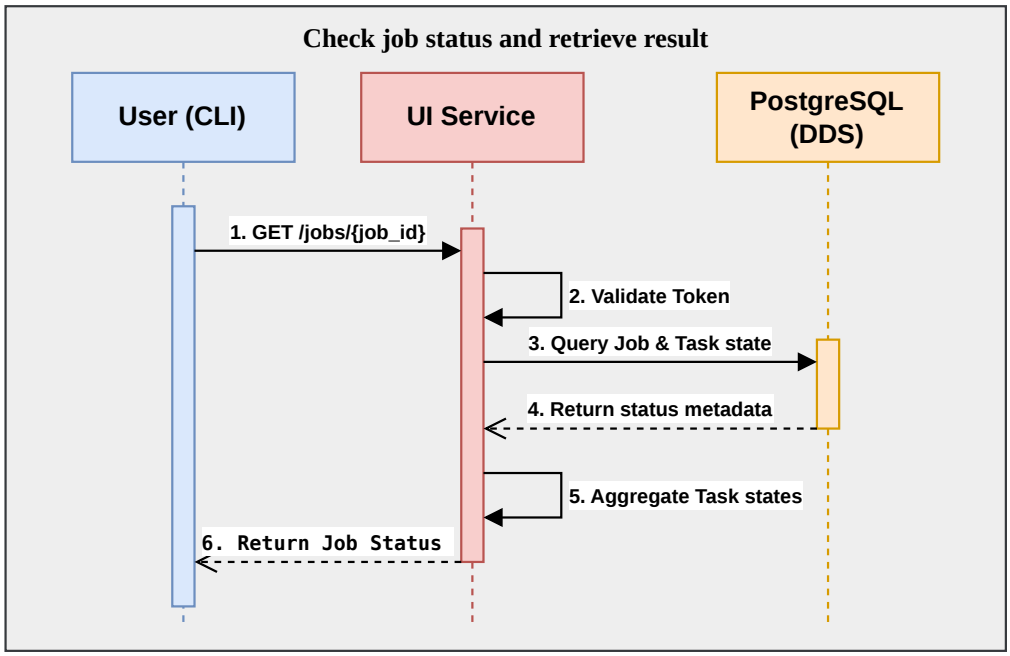
6. Lastly, Manager updates the job in the PostgreSQL as completed, or failed, based on the status of its tasks.



Check job status and retrieve result

User monitors a job's state.

1. User invokes the CLI to check job status.
2. CLI calls the `GET /jobs/{job_id}` on the UI service.
3. UI performs the token validation
4. UI queries PostgreSQL or Manager for job state, which is derived from the states of the tasks under the job. If all the tasks are marked as COMPLETED, then the job is marked as COMPLETED too. In case of failure, we have a separate use case right below.
5. UI returns the status of the job (`SUBMITTED, RUNNING, COMPLETED, FAILED`, etc)

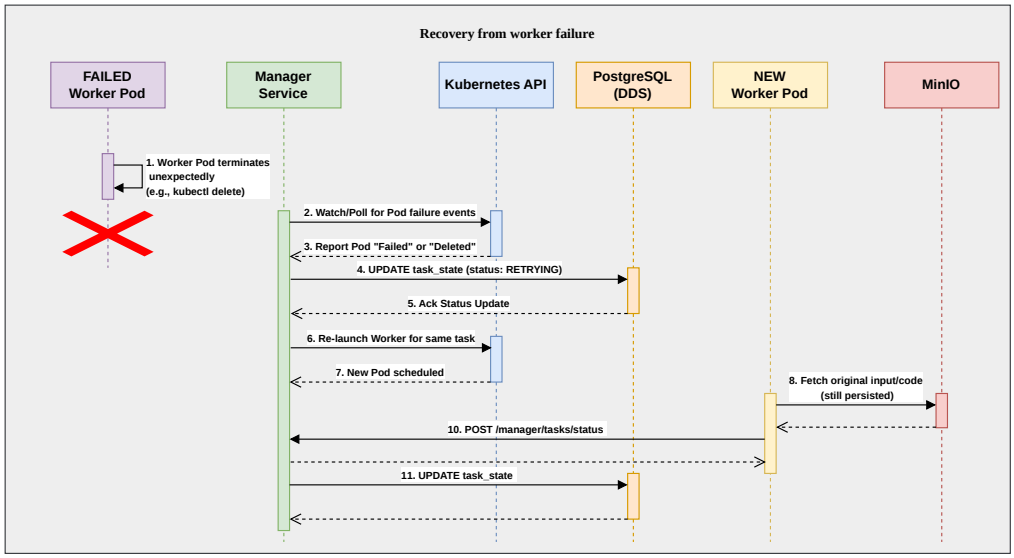


Recovery from worker failure

Our thoughts on this use case were that this will essentially branch from another use case, the Job Execution, since this is the only case where a worker (either mapper or reducer) pod can fail unexpectedly (or expectedly, if done on purpose)

1. During execution of the job, the Manager monitors task progress and worker completion.
2. If a worker pod terminates unexpectedly or doesn't report successful completion, the Manager detects task failure either through Kubernetes pod failure events or missing completion reports.
3. Manager updates task state in PostgreSQL accordingly (e.g. to FAILED, or RETRYING), while the job's status remains RUNNING
4. Since all the input data, codes and intermediate results remain stored in MinIO, the failed task can simply be reassigned
5. Manager launches a new worker pod in K8s, and reassigns the failed mapper/reducer to it
6. The new worker fetches the necessary input from MinIO and re-executes the task.
7. If the retried task completes successfully, the Manager updates the task state in PostgreSQL, and the job continues until completion.
   If for some reason, there's multiple failed retries or recovery is no longer possible, the entire job is marked as FAILED

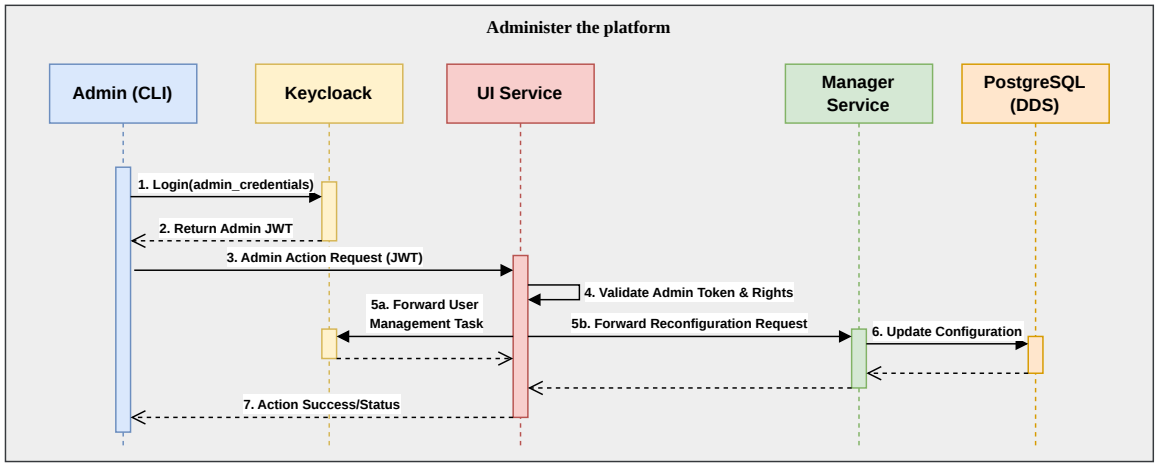
This guarantees us that our system is fault-tolerant. A worker failure doesn't necessarily mean job failure.



Administer the platform

Our assignment refers to the administrator, as a user who can create/delete users, and reconfigure workers.

1. The admin invokes the CLI to perform an administrative action
2. If the admin is not already authenticated, the CLI requests auth through Keycloak, as we have described above.
3. CLI sends the admin request to the UI service, along with the token
4. The UI service confirms validity of token and request
5. If the request concerns user management, then it is passed to Keycloak, which handles user management operations
6. Otherwise, if the request concerns some platform reconfiguration, the UI service forwards it to the Manager, which updates the corresponding configuration in PostgreSQL, and optionally applies it to future job scheduling.



Mapping of Map-Reduce Jobs to Kubernetes Entities

The following table summarizes the mapping between the Map-Reduce system concepts and the corresponding Kubernetes entities:

System Component	**Kubernetes Resource**	**Design Justification**
User Interface (UI)	Deployment	Provides a stateless, replicated entry point.
Manager Service	StatefulSet	Requires stable identity and resiliency for job monitoring.
Workers (Mappers/Reducers)	Job / Pod	Executes finite tasks that must run to completion.

Mapping of System Components to Kubernetes Resources

UI Service (Deployment)

The UI Service will be implemented as a Kubernetes **Deployment** to ensure high availability and horizontal scaling. Since the UI does not store state locally, any pod can handle incoming requests, delegating job tracking to the Manager and Distributed Data Service (DDS).

Manager Service (StatefulSet)

The Manager Service is the central "brain" of the system and is implemented as a **StatefulSet**. Each job is managed by a specific replica pod. If a replica fails, Kubernetes replaces it, and the new pod recovers the job state using information stored in the DDS. The Manager is responsible for invoking the Kubernetes API to create Worker Jobs for the Map, Shuffle, and Reduce phases.

Worker Nodes (Kubernetes Jobs)

The actual parallel computation is performed by **Kubernetes Jobs**. Each Worker runs in a containerized pod to execute mappers and reducers provided by the user. If a worker pod terminates unexpectedly (e.g., via a `kubectl delete` command), the system must detect this and re-assign the task to a new worker to ensure job completion.

Database Schema

We will implement a relational database to store the system's state using PostgreSQL.Our database will contain two tables, Jobs and Tasks.

The Job table stores metadata and status info for each MapReduce Job.

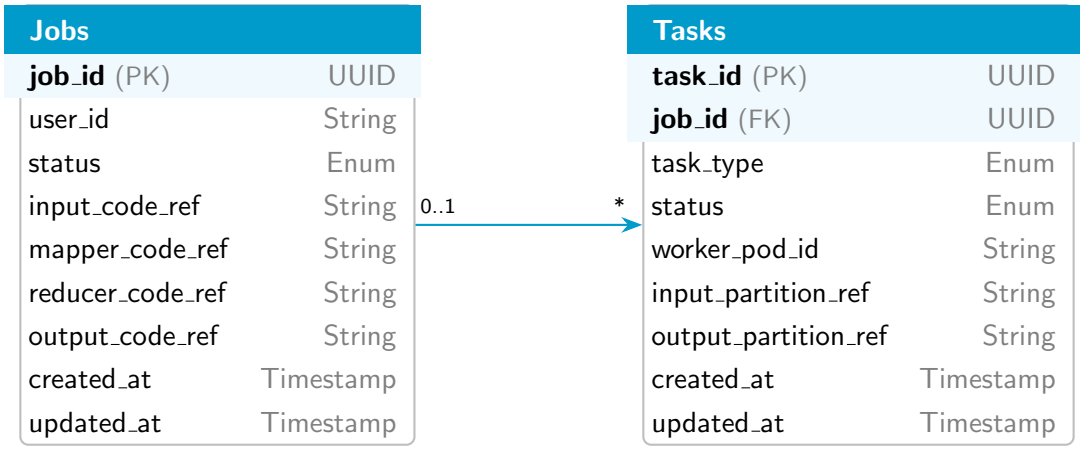
Attributes of this table:

- `job_id` (PK): Unique ID for each job.
- `user_id` (String): ID for the user who submitted. We will extract it from the Keycloak token.
- `status` (Enum): The current state of the Job. The values will be SUBMITTED, RUNNING, COMPLETED and FAILED.
- `input_code_ref` (String): The MinIO object reference to the uploaded data.
- `mapper_code_ref` (String): The MinIO object reference to the mapper script.
- `reducer_code_ref` (String): The MinIO object reference to the reducer script.
- `output_code_ref` (String, Nullable): The MinIO object reference to the final result.
- `created_at / uploaded_at` (Timestamps): Timestamps for tracking when job was submitted or updated.

he Task stores will store info about the tasks that will assigned to Kubernetes pods. Manager will know about the progress of each task and will be able to reassign tasks if a worker fails.

Attributes of this table:

- `task_id` (PK) : Unique ID for a specific id.
- `job_id` (FK) : ID for the parent (job) of the task.
- `task_type` (Enum) : Specifies if this a MAP or a REDUCE task.
- `worker_pod_id` (String, Nullable) : The Kubernetes pod ID of the worker currently executing this task.
- `input_partition_ref` (String): The MinIO object reference to the specific data partition assigned to this task.
- `output_partition_ref` (String, Nullable): The MinIO object reference to the intermediate result (for mappers) or final result (for reducers).
- `created_at / uploaded_at` (Timestamps): Timestamps for tracking when task was submitted or updated.



Database Schema: Jobs and Tasks relationship